

I'll provide you with a comprehensive collection of 500 Python interview questions covering basic to advanced levels, organized by categories with detailed answers to help you succeed in your interviews. [geeksforgeeks+3](#)

Basic Python Questions (1-100)

Python Fundamentals

1. What is Python and why is it popular?

Python is a high-level, interpreted, object-oriented programming language known for its simplicity and readability. It's popular due to its easy syntax, extensive libraries, cross-platform compatibility, strong community support, and versatility across domains like web development, data science, AI/ML, and automation. [inttrvu](#)

2. Is Python compiled or interpreted?

Python is an interpreted language, though it compiles source code to bytecode (.pyc files) before interpretation. The Python interpreter executes bytecode line by line, making it platform-independent. [geeksforgeeks](#)

3. What is PEP 8?

PEP 8 is the Python Enhancement Proposal that provides coding conventions and style guidelines for writing clean, readable Python code. It covers naming conventions, indentation (4 spaces), line length (79 characters), and import organization. [upesonline+1](#)

4. What are Python's key features?

Python is interpreted, dynamically typed, object-oriented, has automatic memory management, extensive standard library, supports multiple programming paradigms, cross-platform compatible, and open-source. [inttrvu](#)

5. What is the difference between Python 2 and Python 3?

Python 3 uses print() as a function vs print statement in Python 2, has better Unicode support, integer division returns float, uses range() instead of xrange(), and improved exception handling syntax with as keyword. [geeksforgeeks](#)

6. What are Python's basic data types?

Python's basic data types include integers (int), floating-point numbers (float), strings (str), booleans (bool), complex numbers (complex), and NoneType. [upesonline+1](#)

7. What is the difference between mutable and immutable objects?

Mutable objects can be modified after creation (lists, dictionaries, sets), while immutable objects cannot be changed (integers, floats, strings, tuples). Attempting to modify immutable objects creates new objects. [datacamp](#)

8. What are lists in Python?

Lists are ordered, mutable collections that can store elements of different data types. They

support indexing, slicing, and various methods like `append()`, `extend()`, `insert()`, `remove()`, and `pop()`. [datacamp+1](#)

9. What are tuples in Python?

Tuples are ordered, immutable collections defined using parentheses. They consume less memory than lists, provide faster iteration, and are commonly used for fixed collections of items. [datacamp+1](#)

10. What is a dictionary in Python?

Dictionaries are unordered collections of key-value pairs defined using curly braces. Keys must be immutable and unique. Dictionaries provide $O(1)$ average-case lookup time. [upesonline](#)

Variables and Operators

11. How do you declare variables in Python?

Python uses dynamic typing, so you don't need explicit type declarations. Simply assign a value: `x = 10`, `name = "John"`. The type is inferred automatically. [inttrvu](#)

12. What is the difference between `==` and `is`?

`==` compares values/contents of objects, while `is` checks if two variables point to the same object in memory (identity comparison). [github](#)

13. What are Python's arithmetic operators?

Addition (+), subtraction (-), multiplication (*), division (/), floor division (//), modulus (%), and exponentiation (**). [geeksforgeeks](#)

14. What are logical operators in Python?

`and`, `or`, and `not` are logical operators used for boolean operations and conditional expressions. [inttrvu](#)

15. What is the difference between `/` and `//`?

`/` performs true division returning float, while `//` performs floor division returning the quotient as an integer. [geeksforgeeks](#)

String Operations

16. How do you concatenate strings in Python?

Use the `+` operator (`"Hello" + "World"`), `join()` method (`"".join(["Hello", "World"])`), or f-strings (`f"{str1}{str2}"`). [geeksforgeeks](#)

17. How do you reverse a string?

Use slicing with negative step: `string[::-1]`. [github](#)

18. What are string methods in Python?

Common methods include `upper()`, `lower()`, `strip()`, `split()`, `replace()`, `find()`, `count()`, `startswith()`, `endswith()`, and `join()`. [upesonline](#)

19. How do you format strings in Python?

Use f-strings (f"Hello {name}"), format() method ("Hello {}".format(name)), or % formatting ("Hello %s" % name). [datacamp](#)

20. How do you count vowels in a string?

```
python
vowels = ['a', 'e', 'i', 'o', 'u']
word = "programming"
count = sum(1 for char in word.lower() if char in vowels)
print(count) # Output: 3
```

Lists and List Operations

21. How do you concatenate two lists?

Use + operator (list1 + list2), extend() method (list1.extend(list2)), or unpacking ([*list1, *list2]). [geeksforgeeks](#)

22. What is list comprehension?

List comprehension provides a concise way to create lists: [x**2 for x in range(10) if x % 2 == 0] creates squares of even numbers. [upesonline](#)

23. How do you remove duplicates from a list?

Convert to set and back: list(set(my_list)) or use dict.fromkeys(): list(dict.fromkeys(my_list)) to preserve order. [upesonline](#)

24. What is the difference between append() and extend()?

append() adds a single element (including another list as one element), while extend() adds all elements from an iterable individually. [datacamp+1](#)

25. How do you sort a list?

Use sort() method for in-place sorting (list.sort()) or sorted() function to return a new sorted list (sorted(list)). [inttrvu](#)

26. What is slicing in Python?

Slicing extracts portions of sequences using syntax [start:stop:step]. Example: list[1:5] gets elements from index 1 to 4. [github](#)

27. How do you find the length of a list?

Use the len() function: len(my_list). [inttrvu](#)

28. What are negative indices in Python?

Negative indices count from the end: -1 is the last element, -2 is second-to-last, etc.. [github](#)

29. How do you copy a list?

Use `list.copy()`, `list[:]`, or `list(original_list)`. Assignment (`new = old`) creates a reference, not a copy. [datacamp](#)

30. What is the difference between `remove()`, `pop()`, and `del`?

`remove()` deletes by value, `pop()` removes by index and returns the element, `del` removes by index or slice without returning. [upesonline](#)

Dictionaries and Sets

31. How do you add elements to a dictionary?

Use assignment: `dict['key'] = value` or `update()` method: `dict.update({'key': value})`. [datacamp+1](#)

32. How do you remove elements from a dictionary?

Use `del dict['key']`, `dict.pop('key')`, or `dict.popitem()` for removing last item. [upesonline](#)

33. What are dictionary methods?

Common methods include `keys()`, `values()`, `items()`, `get()`, `pop()`, `update()`, `clear()`, and `setdefault()`. [inttrvu](#)

34. What is a set in Python?

A set is an unordered collection of unique elements. Created using `{1, 2, 3}` or `set([1, 2, 3])`. Sets support mathematical operations. [upesonline](#)

35. What are set operations?

Union (`|` or `union()`), intersection (`&` or `intersection()`), difference (`-` or `difference()`), symmetric difference (`^` or `symmetric_difference()`) [inttrvu](#).

36. What is the difference between `add()` and `update()` in sets?

`add()` adds a single element, while `update()` adds multiple elements from any iterable. [upesonline](#)

37. How do you iterate through a dictionary?

python

for key **in** dict: *# iterates keys*

for value **in** dict.values(): *# iterates values*

for key, value **in** dict.items(): *# iterates key-value pairs*

38. What is `get()` method in dictionaries?

`get()` returns value for a key or a default value if key doesn't exist, avoiding `KeyError`: `dict.get('key', default_value)`. [inttrvu](#)

39. How do you merge two dictionaries?

Use `update()` method, unpacking (`**dict1, **dict2`), or the `|` operator in Python 3.9+: `dict1 | dict2` [datacamp](#).

40. What is frozenset?

A frozenset is an immutable version of set. Once created, elements cannot be added or removed. Useful as dictionary keys. [upesonline](#)

Control Flow

41. What are conditional statements in Python?

`if`, `elif`, and `else` statements control execution flow based on boolean conditions. [upesonline](#)

42. What is the ternary operator in Python?

Syntax: `value_if_true if condition else value_if_false`. Example: `max = a if a > b else b`. [hipeople](#)

43. What are loops in Python?

Python has `for` loops (for iterating over sequences) and `while` loops (for condition-based iteration). [upesonline](#)

44. What is the difference between `break` and `continue`?

`break` exits the loop entirely, while `continue` skips the current iteration and continues with the next. [upesonline](#)

45. What is the `pass` statement?

`pass` is a null operation placeholder used where syntax requires a statement but no action is needed. [inttrvu](#)

46. What is `range()` function?

`range(start, stop, step)` generates a sequence of numbers. Commonly used in `for` loops: `for i in range(10):`. [inttrvu](#)

47. What is the `else` clause in loops?

The `else` block executes after loop completion if no `break` statement was encountered. [upesonline](#)

48. How do you iterate through multiple lists simultaneously?

Use `zip()` function: `for a, b in zip(list1, list2):`. [inttrvu](#)

49. What is `enumerate()`?

`enumerate()` adds a counter to an iterable: `for index, value in enumerate(list):`. [inttrvu](#)

50. What are nested loops?

Loops inside other loops. Used for multi-dimensional data structures or complex iterations. [upesonline](#)

Functions

51. What is a function in Python?

A function is a reusable block of code that performs a specific task. Defined using def keyword.[pynative+1](#)

52. What are function arguments?

Positional arguments (order matters), keyword arguments (specified by name), default arguments (have default values), and variable-length arguments (*args, **kwargs).[pynative](#)

53. What is the difference between *args and **kwargs?

*args collects variable positional arguments as tuple, **kwargs collects variable keyword arguments as dictionary.[pynative](#)

54. What is a return statement?

return exits a function and optionally returns a value. Without return, functions return None.[upesonline](#)

55. What is function scope?

Variables defined inside functions have local scope. Use global keyword to modify global variables.[pynative](#)

56. What is the difference between local and global variables?

Local variables exist only within functions, global variables are accessible throughout the module.[pynative](#)

57. What is recursion?

A function calling itself to solve problems by breaking them into smaller subproblems. Must have a base case.[github](#)

58. What are lambda functions?

Anonymous functions defined using lambda keyword: lambda x: x**2. Used for short, simple operations.[pynative](#)

59. What are higher-order functions?

Functions that take other functions as arguments or return functions. Examples: map(), filter(), reduce().[github](#)

60. What is map() function?

Applies a function to all items in an iterable: list(map(lambda x: x**2, [1, 2, 3])).[github](#)

61. What is filter() function?

Filters items based on a function that returns True/False: list(filter(lambda x: x > 0, [-1, 0, 1, 2])).[github](#)

62. What is reduce() function?

Cumulatively applies a function to items, reducing to a single value. Available in functools module.[pynative](#)

63. What is function documentation?

Docstrings (triple-quoted strings) document functions, accessible via `__doc__` attribute or `help()` function. [pynative](#)

64. What are default parameter values?

Parameters with default values: `def greet(name="Guest"):`. Used when argument not provided. [pynative](#)

65. What is unpacking in function calls?

Use `*` for list/tuple unpacking, `**` for dictionary unpacking: `func(*args, **kwargs)`. [github](#)

Modules and Packages**66. What is a module in Python?**

A module is a Python file containing definitions, functions, and statements. Imported using `import` statement. [pynative+1](#)

67. How do you import a module?

`import module_name`, `from module import function`, or `import module as alias`. [inttrvu](#)

68. What is `__name__` variable?

`__name__` is `"__main__"` when script runs directly, otherwise it's the module name. Used for conditional execution. [inttrvu](#)

69. What is a package?

A directory containing modules and an `__init__.py` file, organizing related modules hierarchically. [pynative+1](#)

70. What is pip?

Python's package installer for installing and managing third-party libraries from PyPI. [pynative](#)

71. What is a virtual environment?

An isolated Python environment with its own packages and dependencies. Created using `venv` or `virtualenv`. [github](#)

72. What is `__init__.py`?

A file that makes a directory a Python package, can contain initialization code for the package. [pynative](#)

73. What are built-in modules?

Standard library modules like `os`, `sys`, `math`, `datetime`, `random`, `json`, available without installation. [upesonline](#)

74. What is the `sys` module?

Provides system-specific parameters and functions like `sys.argv`, `sys.path`, `sys.exit()`. [github](#)

75. What is the os module?

Provides operating system functionality like file operations, directory navigation, environment variables. [pynative](#)

File Handling

76. How do you open a file in Python?

Use open(filename, mode) where mode is 'r' (read), 'w' (write), 'a' (append), 'b' (binary). [upesonline](#)

77. What is the with statement?

Context manager that ensures proper resource cleanup: with open('file.txt') as f: [inttrvu](#)

78. How do you read a file?

read() reads entire file, readline() reads one line, readlines() returns list of lines. [upesonline](#)

79. How do you write to a file?

Use write() or writelines() methods with file opened in write mode. [upesonline](#)

80. What is the difference between 'w' and 'a' modes?

'w' mode overwrites existing file, 'a' mode appends to existing content. [upesonline](#)

81. How do you check if a file exists?

Use os.path.exists(filename) or os.path.isfile(filename). [github](#)

82. How do you delete a file?

Use os.remove(filename) or os.unlink(filename). [github](#)

83. How do you read CSV files?

Use csv module: import csv; csv.reader(file) or pandas: pd.read_csv('file.csv'). [inttrvu](#)

84. How do you work with JSON files?

Use json module: json.load(file) to read, json.dump(data, file) to write. [upesonline](#)

85. What is binary file handling?

Use 'rb' or 'wb' modes to read/write binary data like images, without text encoding. [upesonline](#)

Exception Handling

86. What is exception handling?

Mechanism to handle runtime errors gracefully using try-except blocks, preventing program crashes. [upesonline](#)

87. What is try-except block?

python

try:

risky code

except ExceptionType:

handle error

88. What is the finally clause?

Code in finally block executes regardless of exceptions, typically for cleanup operations.[upesonline](#)

89. What is the else clause in exception handling?

Executes if no exception occurs in try block.[upesonline](#)

90. How do you raise exceptions?

Use raise statement: raise ValueError("Invalid input").[inttrvu](#)

91. What are common built-in exceptions?

ValueError, TypeError, KeyError, IndexError, FileNotFoundError, ZeroDivisionError, AttributeError.[inttrvu](#)

92. What is custom exception?

User-defined exception class inheriting from Exception class for specific error handling.[upesonline](#)

93. What is assert statement?

Used for debugging, raises AssertionError if condition is False: assert x > 0, "x must be positive".[inttrvu](#)

94. How do you handle multiple exceptions?

python

except (ValueError, TypeError) as e:

handle multiple types

95. What is exception chaining?

Using raise...from to preserve original exception context while raising a new one.[upesonline](#)

Basic Coding Problems

96. Write code to check if a number is prime:

python

def is_prime(n):

if n < 2:

return False

```
for i in range(2, int(n**0.5) + 1):
```

```
    if n % i == 0:
```

```
        return False
```

```
return True
```

97. Write code to find factorial:

python

```
def factorial(n):
```

```
    if n == 0 or n == 1:
```

```
        return 1
```

```
    return n * factorial(n-1)
```

98. Write code to check palindrome:

python

```
def is_palindrome(s):
```

```
    s = s.lower()
```

```
    return s == s[::-1]
```

99. Write code to swap two numbers without temp variable:

python

```
a, b = 5, 10
```

```
a, b = b, a
```

100. Write code to find sum of list elements:

python

```
def sum_list(lst):
```

```
    return sum(lst)
```

Intermediate Python Questions (101-300)

Object-Oriented Programming

101. What is object-oriented programming?

OOP is a programming paradigm based on objects containing data (attributes) and code (methods). Core principles: encapsulation, inheritance, polymorphism, abstraction. [pynative+1](#)

102. What is a class?

A class is a blueprint for creating objects, defining attributes and methods that objects will have.[pynative+1](#)

103. What is an object?

An object is an instance of a class with specific values for attributes defined by the class.[pynative+1](#)

104. What is __init__ method?

The constructor method called when creating a new object, used to initialize object attributes.[interviewbit+1](#)

105. What is self parameter?

self represents the instance of the class, used to access attributes and methods within class methods.[pynative](#)

106. What is inheritance?

Mechanism where a class (child) inherits attributes and methods from another class (parent), promoting code reuse.[hirst+1](#)

107. What are types of inheritance?

Single (one parent), multiple (multiple parents), multilevel (chain), hierarchical (multiple children), hybrid (combination).[hirst](#)

108. What is method overriding?

Child class providing specific implementation of a method already defined in parent class.[pynative](#)

109. What is super() function?

Used to call parent class methods from child class, especially in overridden methods and __init__.[realpython](#)

110. What is encapsulation?

Bundling data and methods together while restricting direct access to some components using access modifiers.[pynative](#)

111. What are access modifiers in Python?

Public (no underscore), protected (single underscore _), private (double underscore __).[realpython](#)

112. What is polymorphism?

Ability of objects to take multiple forms, implementing same method differently in different classes.[hirst+1](#)

113. What is abstraction?

Hiding implementation details and showing only essential features using abstract classes and methods.[pynative](#)

114. What are abstract classes?

Classes with abstract methods (no implementation) that must be implemented by subclasses. Use abc module. [pynative](#)

115. What is method overloading?

Python doesn't support traditional overloading; use default arguments or variable-length arguments instead. [pynative](#)

116. What are class variables vs instance variables?

Class variables are shared by all instances, instance variables are unique to each object. [pynative](#)

117. What are static methods?

Methods that don't access instance or class data, decorated with @staticmethod. [realpython](#)

118. What are class methods?

Methods that receive class as first parameter (cls), decorated with @classmethod. [realpython](#)

119. What is multiple inheritance?

A class inheriting from multiple parent classes, resolved using Method Resolution Order (MRO). [hirst](#)

120. What is Method Resolution Order (MRO)?

The order in which Python looks for methods in inheritance hierarchy, following C3 linearization algorithm. View with Class.__mro__. [indiabix](#)

121. What are magic methods (dunder methods)?

Special methods with double underscores like __init__, __str__, __repr__, __len__, __add__. [realpython](#)

122. What is __str__ vs __repr__?

__str__ returns user-friendly string representation, __repr__ returns developer-friendly, unambiguous representation. [pynative](#)

123. What is operator overloading?

Defining how operators work with custom objects using magic methods like __add__, __sub__. [realpython](#)

124. What is property decorator?

@property creates getter methods, allowing attribute-like access to methods. Use @attribute.setter for setters. [pynative](#)

125. What is composition?

Design principle where a class contains objects of other classes, "has-a" relationship vs inheritance's "is-a". [pynative](#)

Advanced Functions

126. What are closures?

Functions that remember values from enclosing scope even after outer function has finished execution.[hipeople](#)

127. What are decorators?

Functions that modify behavior of other functions without changing their source code, use @decorator syntax.[terminal+1](#)

128. Write a simple decorator:

python

```
def my_decorator(func):  
    def wrapper(*args, **kwargs):  
        print("Before function")  
        result = func(*args, **kwargs)  
        print("After function")  
        return result  
    return wrapper
```

@my_decorator

```
def say_hello():  
    print("Hello!")
```

129. What are decorator arguments?

Decorators that accept parameters require an additional level of nesting.[hipeople](#)

130. What are built-in decorators?

@staticmethod, @classmethod, @property, @abstractmethod.[github](#)

131. What is function caching?

Using @functools.lru_cache to cache function results for memoization.[hipeople](#)

132. What are generator functions?

Functions using yield instead of return, producing values lazily one at a time.[java-success+1](#)

133. What is the yield keyword?

Pauses function execution and returns a value, resuming from that point on next call.[java-success+1](#)

134. What are generators?

Iterators created using generator functions or expressions, memory-efficient for large sequences. [java-success+1](#)

135. What is the difference between generator and normal function?

Generators yield values lazily, maintain state between calls, and are memory-efficient. Normal functions return once and don't maintain state. [hipeople](#)

136. What is generator expression?

Compact syntax for creating generators: `(x**2 for x in range(10))`. [terminal](#)

137. What are iterators?

Objects implementing `__iter__()` and `__next__()` methods for sequential access. [java-success](#)

138. What is the difference between iterator and iterable?

Iterable has `__iter__()` method returning iterator, iterator has both `__iter__()` and `__next__()`. [terminal](#)

139. What is `iter()` and `next()`?

`iter()` returns iterator object, `next()` retrieves next item from iterator. [java-success](#)

140. What is the `StopIteration` exception?

Raised by `__next__()` when no more items available in iterator. [java-success](#)

Advanced Data Structures**141. What is `collections` module?**

Provides specialized container datatypes: `namedtuple`, `deque`, `Counter`, `OrderedDict`, `defaultdict`. [github](#)

142. What is `namedtuple`?

Creates tuple subclasses with named fields for better readability: `Point = namedtuple('Point', ['x', 'y'])`. [github](#)

143. What is `deque`?

Double-ended queue supporting fast `append/pop` from both ends, useful for queues and stacks. [pynative](#)

144. What is `Counter`?

Dictionary subclass for counting hashable objects: `Counter(['a', 'b', 'a'])`. [github](#)

145. What is `defaultdict`?

Dictionary that returns default value for missing keys, avoiding `KeyError`. [pynative](#)

146. What is `OrderedDict`?

Dictionary that maintains insertion order (less relevant since Python 3.7+ dicts are ordered). [pynative](#)

147. What is ChainMap?

Groups multiple dictionaries into single view for searching multiple mappings. [pynative](#)

148. What are heaps in Python?

Binary heap implementation in heapq module for priority queues. [pynative](#)

149. What is bisect module?

Provides binary search and insertion functions for sorted lists. [pynative](#)

150. What is array module?

Efficient storage for homogeneous numeric data, more memory-efficient than lists. [interviewbit](#)

Advanced String Operations**151. What are raw strings?**

Strings prefixed with r, treating backslashes as literal characters: r"\n". [hipeople](#)

152. What are f-strings?

Formatted string literals allowing embedded expressions: f"Value: {x:.2f}". [datacamp](#)

153. What is string interpolation?

Inserting variable values into strings using f-strings, format(), or % operator. [datacamp](#)

154. What are string formatting options?

Width, alignment, padding, precision, type conversion using format specifiers. [datacamp](#)

155. What is join() vs concatenation?

join() is more efficient for multiple strings, concatenation creates intermediate objects. [geeksforgeeks](#)

156. What are regular expressions?

Pattern matching using re module: re.match(), re.search(), re.findall(), re.sub(). [github](#)

157. What are common regex patterns?

\d (digit), \w (word char), \s (whitespace), . (any char), * (0+ times), + (1+ times), ? (0-1 times). [pynative](#)

158. What is re.compile()?

Compiles regex pattern for reuse, improving performance for repeated matching. [pynative](#)

159. What are regex groups?

Capture portions of matched text using parentheses: (\d+)-(\d+). [github](#)

160. What is string encoding?

Converting strings to bytes using encoding like UTF-8: str.encode('utf-8'). [inttrvu](#)

Comprehensions

161. What is dictionary comprehension?

{k: v for k, v in items if condition} creates dictionaries concisely. [upesonline](#)

162. What is set comprehension?

{x for x in iterable if condition} creates sets concisely. [upesonline](#)

163. What are nested comprehensions?

Comprehensions inside comprehensions for multi-dimensional operations. [upesonline](#)

164. What are conditional comprehensions?

Including if-else conditions in comprehensions: [x if x > 0 else 0 for x in lst]. [inttrvu](#)

165. When to use comprehensions vs loops?

Comprehensions for simple transformations, loops for complex logic or multiple operations. [upesonline](#)

Memory Management**166. How does Python manage memory?**

Automatic memory management using private heap, reference counting, and garbage collection. [terminal](#)

167. What is reference counting?

Python tracks number of references to objects, deallocating when count reaches zero. [terminal](#)

168. What is garbage collection?

Automatic reclamation of memory from objects no longer referenced, handles circular references. [terminal](#)

169. What is del statement?

Removes variable binding, decreases reference count, but doesn't necessarily delete object immediately. [upesonline](#)

170. What is gc module?

Interface to garbage collector, allowing manual control: gc.collect(), gc.disable(). [github](#)

171. What are weak references?

References that don't increase reference count, allowing garbage collection. Use weakref module. [pynative](#)

172. What is memory profiling?

Analyzing memory usage using tools like memory_profiler, tracemalloc. [github](#)

173. What are memory leaks?

Objects remaining in memory due to unintended references, preventing garbage collection. [pynative](#)

174. What is `__del__` method?

Destructor called when object is about to be destroyed, but timing is unpredictable. [pynative](#)

175. What is interning?

Python caching small integers and strings for memory efficiency and performance. [inttrvu](#)

Advanced List Operations

176. What is `zip()` function?

Combines multiple iterables into iterator of tuples: `list(zip([1,2], ['a','b']))`. [inttrvu](#)

177. What is `enumerate()` with start parameter?

`enumerate(iterable, start=1)` begins counting from specified number. [upesonline](#)

178. What is `reversed()` function?

Returns reverse iterator without modifying original: `list(reversed([1,2,3]))`. [inttrvu](#)

179. What is `sorted()` with key parameter?

`sorted(lst, key=lambda x: x[1])` sorts using custom comparison function. [upesonline](#)

180. What is `all()` and `any()`?

`all()` returns True if all elements are truthy, `any()` returns True if at least one is truthy. [pynative](#)

181. What is list unpacking?

`a, b, *rest = [1, 2, 3, 4, 5]` assigns first two to a and b, rest to rest. [pynative](#)

182. What are shallow vs deep copies?

Shallow copy (`copy()`) copies references, deep copy (`deepcopy()`) copies nested objects recursively. [pynative](#)

183. What is copy module?

Provides `copy()` for shallow copies and `deepcopy()` for deep copies. [pynative](#)

184. What is flattening a nested list?

python

```
nested = [[1,2], [3,4]]
```

```
flat = [item for sublist in nested for item in sublist]
```

185. What is list rotation?

`lst = lst[n:] + lst[:n]` rotates list by n positions. [github](#)

Advanced Dictionary Operations

186. What is dictionary unpacking?

`{**dict1, **dict2}` merges dictionaries, with dict2 values overwriting dict1. [datacamp](#)

187. What is.setdefault() method?

Returns value if key exists, otherwise sets and returns default: dict.setdefault('key', default).[inttrvu](#)

188. What is dictionary view objects?

keys(), values(), items() return view objects reflecting dictionary changes.[upesonline](#)

189. What is dictionary ordering?

Since Python 3.7+, dictionaries maintain insertion order as part of language specification.[pynative](#)

190. How to sort dictionary by values?

sorted(dict.items(), key=lambda x: x[1]) returns sorted list of tuples.[upesonline](#)

191. What is dictionary filtering?

{k: v for k, v in dict.items() if condition} creates filtered dictionary.[upesonline](#)

192. What is dictionary merging with update()?

dict1.update(dict2) modifies dict1 in-place, adding/updating keys from dict2.[upesonline](#)

193. How to invert a dictionary?

{v: k for k, v in dict.items()} swaps keys and values.[upesonline](#)

194. What is nested dictionary access?

Use chained get() or try-except to safely access nested keys.[upesonline](#)

195. What is dictionary key requirements?

Keys must be hashable (immutable): strings, numbers, tuples of immutables.[upesonline](#)

Functional Programming**196. What is functional programming?**

Programming paradigm treating computation as evaluation of functions, avoiding state changes.[hipeople](#)

197. What are first-class functions?

Functions as objects that can be assigned to variables, passed as arguments, returned from functions.[hipeople](#)

198. What is pure function?

Function with no side effects, same input always produces same output.[hipeople](#)

199. What are side effects?

Modifications to state outside function scope: modifying globals, I/O operations.[hipeople](#)

200. What is immutability?

Objects that cannot be modified after creation, reducing bugs and enabling optimization.[hipeople](#)

201. What is functools module?

Provides higher-order functions: reduce(), partial(), lru_cache(), wraps().[github](#)

202. What is partial() function?

Creates new function with some arguments pre-filled: from functools import partial.[github](#)

203. What is wraps() decorator?

Preserves original function's metadata when creating decorators.[hipeople](#)

204. What is function composition?

Combining functions where output of one becomes input of another.[hipeople](#)

205. What is currying?

Transforming function with multiple arguments into sequence of single-argument functions.[hipeople](#)

Advanced Exception Handling**206. What is exception hierarchy?**

All exceptions inherit from BaseException, most from Exception. Custom exceptions should inherit from Exception.[inttrvu](#)

207. What is try-except-else-finally?

Complete exception handling structure: try (attempt), except (handle error), else (no error), finally (always).[upesonline](#)

208. What is exception context?

Using as keyword to capture exception object: except ValueError as e:.[inttrvu](#)

209. What are context managers?

Objects implementing __enter__() and __exit__() for resource management.[upesonline](#)

210. What is contextlib module?

Provides utilities for working with context managers: @contextmanager decorator.[pynative](#)

211. How to create custom context manager?

Implement __enter__() and __exit__() methods or use @contextmanager decorator.[upesonline](#)

212. What is exception suppression?

Using contextlib.suppress() to ignore specific exceptions.[pynative](#)

213. What is ExceptionGroup?

Python 3.11+ feature for handling multiple exceptions simultaneously.[pynative](#)

214. What is traceback?

Stack trace showing exception origin, accessed via traceback module.[pynative](#)

215. How to log exceptions?

Use logging module with `logger.exception()` to log with traceback. [pynative](#)

Datetime and Time**216. What is datetime module?**

Provides classes for working with dates and times: `date`, `time`, `datetime`, `timedelta`. [github](#)

217. How to get current date/time?

`datetime.datetime.now()` returns current date and time. [pynative](#)

218. What is timedelta?

Represents duration/difference between dates or times. [pynative](#)

219. How to format datetime?

Use `strftime()`: `dt.strftime('%Y-%m-%d %H:%M:%S')`. [github](#)

220. How to parse datetime strings?

Use `strptime()`: `datetime.strptime('2025-11-17', '%Y-%m-%d')`. [github](#)

221. What is timezone handling?

Use `pytz` or `zoneinfo` modules for timezone-aware datetime objects. [pynative](#)

222. What is time module?

Provides time-related functions: `time.time()`, `time.sleep()`, `time.strftime()`. [github](#)

223. How to measure execution time?

Use `time.perf_counter()` or `timeit` module for accurate timing. [pynative](#)

224. What is Unix timestamp?

Seconds since January 1, 1970, obtained with `time.time()` or `datetime.timestamp()`. [github](#)

225. What is calendar module?

Provides calendar-related functions like determining leap years, month calendars. [pynative](#)

Testing and Debugging**226. What is unit testing?**

Testing individual components in isolation using `unittest` or `pytest` frameworks. [pynative](#)

227. What is unittest module?

Python's built-in testing framework with test cases, fixtures, and assertions. [pynative](#)

228. What is pytest?

Popular third-party testing framework with simpler syntax and powerful features. [pynative](#)

229. What are assertions?

Statements checking conditions: `assert x > 0`, `self.assertEqual(a, b)`. [github](#)

230. What are test fixtures?

Setup and teardown code for tests using setUp(), tearDown(), or pytest fixtures. [pynative](#)

231. What is mocking?

Replacing real objects with mock objects for testing using unittest.mock. [github](#)

232. What is code coverage?

Measuring percentage of code executed by tests using coverage.py. [github](#)

233. What is debugging?

Finding and fixing code errors using debuggers, print statements, logging. [upesonline](#)

234. What is pdb module?

Python's built-in debugger for interactive debugging: import pdb; pdb.set_trace(). [github](#)

235. What are breakpoints?

Points where execution pauses for inspection, set with breakpoint() in Python 3.7+. [pynative](#)

Working with APIs and JSON**236. What is JSON?**

JavaScript Object Notation, lightweight data interchange format. [upesonline](#)

237. How to parse JSON?

Use json.loads() for strings, json.load() for files. [upesonline](#)

238. How to create JSON?

Use json.dumps() for strings, json.dump() for files. [upesonline](#)

239. What is JSON serialization?

Converting Python objects to JSON format. [upesonline](#)

240. What is JSON deserialization?

Converting JSON to Python objects. [upesonline](#)

241. How to handle custom objects in JSON?

Implement custom encoder/decoder using json.JSONEncoder and object_hook. [inttrvu](#)

242. What is requests library?

Third-party library for making HTTP requests: GET, POST, PUT, DELETE. [pynative](#)

243. How to make GET request?

import requests; response = requests.get(url). [github](#)

244. How to make POST request?

requests.post(url, data=data, json=json_data). [github](#)

245. How to handle API responses?

Check response.status_code, access response.json() or response.text. [github](#)

Data Processing

246. What is list slicing with step?

lst[start:stop:step] where step can be negative for reverse: lst[::-1].[hirst](#)

247. How to remove whitespace from strings?

Use strip() (both ends), lstrip() (left), rstrip() (right).[upesonline](#)

248. How to split strings?

str.split(delimiter) splits into list, str.splitlines() splits by newlines.[upesonline](#)

249. How to check string content?

isdigit(), isalpha(), isalnum(), isspace(), isupper(), islower().[inttrvu](#)

250. What is string translation?

Using str.translate() with translation table for character mapping.[upesonline](#)

251. How to find substring?

str.find(sub) returns index or -1, str.index(sub) raises ValueError if not found.[upesonline](#)

252. What is string partition?

str.partition(sep) splits into 3-tuple: before, separator, after.[upesonline](#)

253. How to pad strings?

str.ljust(width), str.rjust(width), str.center(width), str.zfill(width).[inttrvu](#)

254. What is string capitalization?

capitalize(), title(), upper(), lower(), swapcase().[inttrvu](#)

255. How to count substring occurrences?

str.count(substring, start, end) returns number of occurrences.[github](#)

Algorithm Problems

256. Write Fibonacci sequence:

python

```
def fibonacci(n):
```

```
    a, b = 0, 1
```

```
    for _ in range(n):
```

```
        yield a
```

```
        a, b = b, a + b
```

257. Write binary search:

python

```
def binary_search(arr, target):
```

```
    left, right = 0, len(arr) - 1
```

```
    while left <= right:
```

```
        mid = (left + right) // 2
```

```
        if arr[mid] == target:
```

```
            return mid
```

```
        elif arr[mid] < target:
```

```
            left = mid + 1
```

```
        else:
```

```
            right = mid - 1
```

```
    return -1
```

258. Write bubble sort:

python

```
def bubble_sort(arr):
```

```
    n = len(arr)
```

```
    for i in range(n):
```

```
        for j in range(0, n-i-1):
```

```
            if arr[j] > arr[j+1]:
```

```
                arr[j], arr[j+1] = arr[j+1], arr[j]
```

```
    return arr
```

259. Write merge sort:

python

```
def merge_sort(arr):
```

```
    if len(arr) <= 1:
```

```
        return arr
```

```
    mid = len(arr) // 2
```

```
    left = merge_sort(arr[:mid])
```

```
    right = merge_sort(arr[mid:])
```

```
    return merge(left, right)
```

260. Write quick sort:

python

```
def quick_sort(arr):  
    if len(arr) <= 1:  
        return arr  
  
    pivot = arr[len(arr) // 2]  
  
    left = [x for x in arr if x < pivot]  
    middle = [x for x in arr if x == pivot]  
    right = [x for x in arr if x > pivot]  
  
    return quick_sort(left) + middle + quick_sort(right)
```

261. Find maximum subarray sum (Kadane's algorithm):

python

```
def max_subarray_sum(arr):  
    max_sum = current_sum = arr[0]  
  
    for num in arr[1:]:  
        current_sum = max(num, current_sum + num)  
        max_sum = max(max_sum, current_sum)  
  
    return max_sum
```

262. Two sum problem:

python

```
def two_sum(nums, target):  
    seen = {}  
  
    for i, num in enumerate(nums):  
        complement = target - num  
        if complement in seen:  
            return [seen[complement], i]  
        seen[num] = i
```

```
    return []
```

263. Remove duplicates from sorted array:

python

```
def remove_duplicates(nums):  
    if not nums:  
        return 0  
    write_index = 1  
    for i in range(1, len(nums)):  
        if nums[i] != nums[i-1]:  
            nums[write_index] = nums[i]  
            write_index += 1  
    return write_index
```

264. Valid parentheses:

python

```
def is_valid_parentheses(s):  
    stack = []  
    mapping = {')': '(', '}': '{', ']': '['}  
    for char in s:  
        if char in mapping:  
            if not stack or stack.pop() != mapping[char]:  
                return False  
        else:  
            stack.append(char)  
    return not stack
```

265. Find missing number in array:

python

```
def find_missing(nums):  
    n = len(nums)
```

```
expected_sum = n * (n + 1) // 2
actual_sum = sum(nums)

return expected_sum - actual_sum
```

266. Rotate array:

python

```
def rotate(nums, k):
    k = k % len(nums)
    nums[:] = nums[-k:] + nums[:-k]
```

267. Longest common prefix:

python

```
def longest_common_prefix(strs):
    if not strs:
        return ""
    prefix = strs[0]
    for s in strs[1:]:
        while not s.startswith(prefix):
            prefix = prefix[:-1]
        if not prefix:
            return ""
    return prefix
```

268. Reverse linked list:

python

```
def reverse_linked_list(head):
    prev = None
    current = head
    while current:
        next_node = current.next
        current.next = prev
        prev = current
        current = next_node
```

```
    prev = current
    current = next_node

    return prev
```

269. Find cycle in linked list:

python

```
def has_cycle(head):
    slow = fast = head

    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next

    if slow == fast:
        return True

    return False
```

270. Implement queue using stacks:

python

```
class QueueUsingStacks:
    def __init__(self):
        self.s1 = []
        self.s2 = []

    def enqueue(self, x):
        self.s1.append(x)

    def dequeue(self):
        if not self.s2:
            while self.s1:
                self.s2.append(self.s1.pop())

        return self.s2.pop() if self.s2 else None
```

271. Implement stack using queues:

python

```
from collections import deque
```

```
class StackUsingQueue:
```

```
    def __init__(self):
```

```
        self.q = deque()
```

```
    def push(self, x):
```

```
        self.q.append(x)
```

```
        for _ in range(len(self.q) - 1):
```

```
            self.q.append(self.q.popleft())
```

```
    def pop(self):
```

```
        return self.q.popleft()
```

272. Find first non-repeating character:

python

```
def first_non_repeating(s):
```

```
    from collections import Counter
```

```
    count = Counter(s)
```

```
    for i, char in enumerate(s):
```

```
        if count[char] == 1:
```

```
            return i
```

```
    return -1
```

273. Group anagrams:

python

```
def group_anagrams(strs):
```

```
    from collections import defaultdict
```

```

groups = defaultdict(list)

for s in strs:
    key = ".join(sorted(s))
    groups[key].append(s)

return list(groups.values())

```

274. Valid anagram:

python

```

def is_anagram(s, t):

    from collections import Counter

    return Counter(s) == Counter(t)

```

275. Merge two sorted lists:

python

```

def merge_sorted_lists(l1, l2):

    result = []

    i = j = 0

    while i < len(l1) and j < len(l2):

        if l1[i] < l2[j]:

            result.append(l1[i])

            i += 1

        else:

            result.append(l2[j])

            j += 1

    result.extend(l1[i:])

    result.extend(l2[j:])

    return result

```

276. Find peak element:

python

```

def find_peak(nums):

```

```

left, right = 0, len(nums) - 1

while left < right:
    mid = (left + right) // 2
    if nums[mid] < nums[mid + 1]:
        left = mid + 1
    else:
        right = mid
return left

```

277. Search in rotated sorted array:

python

```

def search_rotated(nums, target):
    left, right = 0, len(nums) - 1

    while left <= right:
        mid = (left + right) // 2
        if nums[mid] == target:
            return mid
        if nums[left] <= nums[mid]:
            if nums[left] <= target < nums[mid]:
                right = mid - 1
            else:
                left = mid + 1
        else:
            if nums[mid] < target <= nums[right]:
                left = mid + 1
            else:
                right = mid - 1
    return -1

```

278. Container with most water:

python

```
def max_area(height):
```

```
    left, right = 0, len(height) - 1
```

```
    max_water = 0
```

```
    while left < right:
```

```
        width = right - left
```

```
        max_water = max(max_water, min(height[left], height[right]) * width)
```

```
        if height[left] < height[right]:
```

```
            left += 1
```

```
        else:
```

```
            right -= 1
```

```
    return max_water
```

279. Product of array except self:

python

```
def product_except_self(nums):
```

```
    n = len(nums)
```

```
    result = [1] * n
```

```
    left = 1
```

```
    for i in range(n):
```

```
        result[i] *= left
```

```
        left *= nums[i]
```

```
    right = 1
```

```
    for i in range(n-1, -1, -1):
```

```
        result[i] *= right
```

```
        right *= nums[i]
```

```
    return result
```

280. Longest consecutive sequence:

python

```

def longest_consecutive(nums):
    num_set = set(nums)
    longest = 0
    for num in num_set:
        if num - 1 not in num_set:
            current = num
            streak = 1
            while current + 1 in num_set:
                current += 1
                streak += 1
            longest = max(longest, streak)
    return longest

```

281. Climbing stairs (Fibonacci variant):

python

```

def climb_stairs(n):
    if n <= 2:
        return n
    a, b = 1, 2
    for _ in range(n - 2):
        a, b = b, a + b
    return b

```

282. Best time to buy and sell stock:

python

```

def max_profit(prices):
    min_price = float('inf')
    max_profit = 0
    for price in prices:
        min_price = min(min_price, price)

```

```
max_profit = max(max_profit, price - min_price)
```

```
return max_profit
```

283. House robber:

python

```
def rob(nums):
```

```
    if not nums:
```

```
        return 0
```

```
    if len(nums) == 1:
```

```
        return nums[0]
```

```
    prev1, prev2 = 0, 0
```

```
    for num in nums:
```

```
        prev1, prev2 = max(prev1, prev2 + num), prev1
```

```
    return prev1
```

284. Coin change problem:

python

```
def coin_change(coins, amount):
```

```
    dp = [float('inf')] * (amount + 1)
```

```
    dp[0] = 0
```

```
    for coin in coins:
```

```
        for i in range(coin, amount + 1):
```

```
            dp[i] = min(dp[i], dp[i - coin] + 1)
```

```
    return dp[amount] if dp[amount] != float('inf') else -1
```

285. Longest increasing subsequence:

python

```
def longest_increasing_subsequence(nums):
```

```
    if not nums:
```

```
        return 0
```

```
    dp = [1] * len(nums)
```

```

for i in range(1, len(nums)):
    for j in range(i):
        if nums[i] > nums[j]:
            dp[i] = max(dp[i], dp[j] + 1)
return max(dp)

```

286. Word break:

python

```

def word_break(s, wordDict):
    dp = [False] * (len(s) + 1)
    dp[0] = True
    for i in range(1, len(s) + 1):
        for j in range(i):
            if dp[j] and s[j:i] in wordDict:
                dp[i] = True
                break
    return dp[len(s)]

```

287. Implement Trie:

python

```

class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_end = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word):

```

```
node = self.root  
  
for char in word:  
    if char not in node.children:  
        node.children[char] = TrieNode()  
    node = node.children[char]  
  
node.is_end = True
```

```
def search(self, word):  
    node = self.root  
  
    for char in word:  
        if char not in node.children:  
            return False  
        node = node.children[char]  
  
    return node.is_end
```

288. Serialize and deserialize binary tree:

python

```
class TreeNode:  
    def __init__(self, val=0):  
        self.val = val  
        self.left = None  
        self.right = None  
  
def serialize(root):  
    if not root:  
        return "None"  
    return f"{root.val},{serialize(root.left)},{serialize(root.right)}"  
  
def deserialize(data):
```

```

def helper(nodes):
    val = next(nodes)
    if val == "None":
        return None
    node = TreeNode(int(val))
    node.left = helper(nodes)
    node.right = helper(nodes)
    return node
return helper(iter(data.split(',')))

```

289. Level order traversal:

python

```

def level_order(root):
    from collections import deque
    if not root:
        return []
    result, queue = [], deque([root])
    while queue:
        level = []
        for _ in range(len(queue)):
            node = queue.popleft()
            level.append(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        result.append(level)
    return result

```

290. Maximum depth of binary tree:

python

```
def max_depth(root):  
    if not root:  
        return 0  
  
    return 1 + max(max_depth(root.left), max_depth(root.right))
```

291. Validate binary search tree:

python

```
def is_valid_bst(root, min_val=float('-inf'), max_val=float('inf')):  
    if not root:  
        return True  
  
    if not (min_val < root.val < max_val):  
        return False  
  
    return (is_valid_bst(root.left, min_val, root.val) and  
            is_valid_bst(root.right, root.val, max_val))
```

292. Lowest common ancestor:

python

```
def lowest_common_ancestor(root, p, q):  
    if not root or root == p or root == q:  
        return root  
  
    left = lowest_common_ancestor(root.left, p, q)  
    right = lowest_common_ancestor(root.right, p, q)  
  
    if left and right:  
        return root  
  
    return left or right
```

293. Path sum:

python

```
def has_path_sum(root, target_sum):  
    if not root:
```

```

    return False

if not root.left and not root.right:
    return root.val == target_sum

return (has_path_sum(root.left, target_sum - root.val) or
        has_path_sum(root.right, target_sum - root.val))

```

294. Number of islands:

python

```

def num_islands(grid):
    if not grid:
        return 0

    def dfs(i, j):
        if (i < 0 or i >= len(grid) or j < 0 or j >= len(grid[0]) or
            grid[i][j] != '1'):
            return
        grid[i][j] = '0'
        dfs(i+1, j)
        dfs(i-1, j)
        dfs(i, j+1)
        dfs(i, j-1)

    count = 0
    for i in range(len(grid)):
        for j in range(len(grid[0])):
            if grid[i][j] == '1':
                dfs(i, j)
                count += 1

    return count

```

295. Course schedule (cycle detection):

python

```
def can_finish(num_courses, prerequisites):  
    from collections import defaultdict, deque  
    graph = defaultdict(list)  
    in_degree = [0] * num_courses  
  
    for course, prereq in prerequisites:  
        graph[prereq].append(course)  
        in_degree[course] += 1  
  
    queue = deque([i for i in range(num_courses) if in_degree[i] == 0])  
    count = 0  
  
    while queue:  
        course = queue.popleft()  
        count += 1  
        for next_course in graph[course]:  
            in_degree[next_course] -= 1  
            if in_degree[next_course] == 0:  
                queue.append(next_course)  
  
    return count == num_courses
```

296. Clone graph:

python

```
def clone_graph(node):  
    if not node:  
        return None
```

```

visited = {}

def dfs(node):
    if node in visited:
        return visited[node]

    clone = Node(node.val)
    visited[node] = clone

    for neighbor in node.neighbors:
        clone.neighbors.append(dfs(neighbor))

    return clone

return dfs(node)

```

297. Permutations:

python

```

def permute(nums):
    result = []

    def backtrack(path, remaining):
        if not remaining:
            result.append(path[:])
            return

        for i in range(len(remaining)):
            backtrack(path + [remaining[i]], remaining[:i] + remaining[i+1:])

    backtrack([], nums)

    return result

```

298. Combinations:

python

```

def combine(n, k):
    result = []

    def backtrack(start, path):
        if len(path) == k:

```

```

        result.append(path[:])

    return

    for i in range(start, n + 1):
        path.append(i)
        backtrack(i + 1, path)
        path.pop()

    backtrack(1, [])

    return result

```

299. Subsets:

python

```

def subsets(nums):
    result = []

    def backtrack(start, path):
        result.append(path[:])

        for i in range(start, len(nums)):
            path.append(nums[i])
            backtrack(i + 1, path)
            path.pop()

    backtrack(0, [])

    return result

```

300. Generate parentheses:

python

```

def generate_parentheses(n):
    result = []

    def backtrack(s, left, right):
        if len(s) == 2 * n:
            result.append(s)

        return

```

```

if left < n:
    backtrack(s + '(', left + 1, right)
if right < left:
    backtrack(s + ')', left, right + 1)
backtrack("", 0, 0)

return result

```

Advanced Python Questions (301-500)

Concurrency and Multithreading

301. What is concurrency?

Multiple tasks making progress without necessarily running simultaneously, enabling efficient resource usage. [pynative](#)

302. What is the difference between concurrency and parallelism?

Concurrency is about dealing with multiple tasks, parallelism is about executing multiple tasks simultaneously. [pynative](#)

303. What is the Global Interpreter Lock (GIL)?

Python's mutex preventing multiple native threads from executing Python bytecode simultaneously, affecting CPU-bound multithreading. [pynative](#)

304. What is threading in Python?

threading module provides thread-based parallelism for I/O-bound tasks, but limited by GIL for CPU-bound tasks. [pynative](#)

305. How to create a thread?

```

python

import threading

t = threading.Thread(target=function, args=(arg1,))

t.start()

t.join()

```

306. What are thread synchronization mechanisms?

Lock, RLock, Semaphore, Event, Condition for coordinating threads and preventing race conditions. [pynative](#)

307. What is multiprocessing?

multiprocessing module creates separate processes bypassing GIL, suitable for CPU-bound tasks. [pynative](#)

308. How to create a process?

python

from multiprocessing **import** Process

p = Process(target=function, args=(arg1,))

p.start()

p.join()

309. What is process pool?

multiprocessing.Pool manages worker processes for parallel task execution with map, apply methods.[pynative](#)

310. What is asyncio?

Python's library for asynchronous programming using async/await syntax for concurrent I/O operations.[pynative](#)

311. What are coroutines?

Functions defined with async def, pausable and resumable using await keyword.[pynative](#)

312. What is async and await?

async defines coroutine function, await pauses coroutine until awaited operation completes.[pynative](#)

313. What is event loop?

Core of asyncio managing and executing asynchronous tasks: asyncio.run(), asyncio.get_event_loop().[github](#)

314. What are futures and tasks?

Future represents eventual result, Task wraps coroutine for scheduling in event loop.[pynative](#)

315. What is concurrent.futures?

High-level interface for asynchronous execution using ThreadPoolExecutor and ProcessPoolExecutor.[pynative](#)

316. What is thread-safe queue?

queue.Queue provides thread-safe FIFO queue for producer-consumer patterns.[pynative](#)

317. What is deadlock?

Situation where threads wait for each other indefinitely, preventing progress.[pynative](#)

318. What is race condition?

Bug when outcome depends on uncontrolled timing of events, requiring synchronization.[pynative](#)

319. What is daemon thread?

Background thread that doesn't prevent program exit: `thread.daemon = True`. [github](#)

320. What is thread pool?

python

```
from concurrent.futures import ThreadPoolExecutor
```

```
with ThreadPoolExecutor(max_workers=5) as executor:
```

```
    results = executor.map(function, items)
```

Advanced OOP Concepts

321. What is metaclass?

Class of a class, defining how classes behave. Created using type or custom metaclass. [pynative](#)

322. How to create metaclass?

python

```
class Meta(type):
```

```
    def __new__(cls, name, bases, dct):
```

```
        return super().__new__(cls, name, bases, dct)
```

```
class MyClass(metaclass=Meta):
```

```
    pass
```

323. What is __new__ vs __init__?

`__new__` creates instance (class method), `__init__` initializes instance (instance method). [pynative](#)

324. What is singleton pattern?

Design pattern ensuring class has only one instance, implemented using metaclass or class variable. [pynative](#)

325. What is factory pattern?

Design pattern creating objects without specifying exact class, using factory method. [pynative](#)

326. What is abstract base class (ABC)?

python

```
from abc import ABC, abstractmethod
```

```
class Shape(ABC):  
    @abstractmethod  
    def area(self):  
        pass
```

327. What is `isinstance()` vs `type()`?

`isinstance()` considers inheritance, `type()` checks exact type. [pynative](#)

328. What is duck typing?

"If it walks like a duck and quacks like a duck, it's a duck" - focus on behavior, not type. [pynative](#)

329. What is protocol in Python?

Informal interface defined by methods, supporting structural subtyping (Python 3.8+ `typing.Protocol`). [pynative](#)

330. What is mixin?

Class providing methods to other classes through multiple inheritance without being base class. [pynative](#)

331. What is descriptor?

Object defining `__get__`, `__set__`, `__delete__` for attribute access control. [pynative](#)

332. What is property vs descriptor?

property is high-level descriptor implementation for simple getter/setter/deleter. [pynative](#)

333. What is slots?

`__slots__` restricts instance attributes, reducing memory and improving access speed. [pynative](#)

334. What is `__call__` method?

Makes instance callable like function: `obj(args)`. [realpython](#)

335. What is `__getattr__` vs `__getattribute__`?

`__getattr__` called when attribute not found, `__getattribute__` called for every attribute access. [pynative](#)

336. What is `__setattr__` and `__delattr__`?

Customize attribute assignment and deletion behavior. [pynative](#)

337. What is `__enter__` and `__exit__`?

Define context manager protocol for with statement. [upesonline](#)

338. What is `__iter__` and `__next__`?

Make object iterable by implementing iterator protocol. [java-success](#)

339. What is `__contains__`?

Implements in operator: item in container. [realpython](#)

340. What is `__eq__`, `__lt__`, etc.?

Comparison magic methods for `==`, `<`, `>`, `<=`, `>=`, `!=` operators. [pynative](#)

Performance Optimization

341. What is profiling?

Analyzing code execution time and resource usage using cProfile, line_profiler. [github](#)

342. How to profile Python code?

```
python
```

```
import cProfile
```

```
cProfile.run('function()')
```

343. What is timeit module?

Measures small code snippet execution time accurately: `timeit.timeit('code', number=1000)`. [github](#)

344. What are optimization techniques?

Use built-in functions, list comprehensions, generators, appropriate data structures, avoid global lookups. [pynative](#)

345. What is memoization?

Caching function results to avoid recomputation: `@functools.lru_cache()`. [github](#)

346. What is lazy evaluation?

Delaying computation until value is needed, using generators and iterators. [java-success](#)

347. What is vectorization?

Performing operations on entire arrays instead of loops, using NumPy. [pynative](#)

348. What is just-in-time (JIT) compilation?

Compiling code at runtime for performance, using tools like PyPy or Numba. [pynative](#)

349. What is Cython?

Python superset compiled to C for performance, supporting type declarations. [pynative](#)

350. What is NumPy?

Library for numerical computing with efficient array operations and mathematical functions. [pynative](#)

351. Why is NumPy faster?

Uses optimized C code, contiguous memory, vectorized operations, avoiding Python overhead. [pynative](#)

352. What is list vs NumPy array?

Lists store references, arrays store data contiguously with fixed type, enabling faster operations. [pynative](#)

353. What is __slots__ optimization?

Reduces memory by using fixed attributes instead of dictionary, improving access speed. [pynative](#)

354. What is string interning?

Python caches small strings and identifiers for memory efficiency and comparison speed. [inttrvu](#)

355. What is copy-on-write?

Optimization delaying copying until modification occurs. [pynative](#)

356. How to optimize loops?

Move invariant code outside, use comprehensions, consider vectorization, minimize function calls. [pynative](#)

357. What are common performance pitfalls?

String concatenation in loops, deep nesting, unnecessary copying, global variable access. [pynative](#)

358. What is algorithmic complexity?

Big O notation describing performance characteristics: $O(1)$, $O(\log n)$, $O(n)$, $O(n^2)$. [github](#)

359. What is space complexity?

Memory usage analysis of algorithms, considering auxiliary space. [github](#)

360. What is trade-off between time and space?

Optimization often involves balancing computation time against memory usage. [github](#)

Web Development**361. What is Flask?**

Lightweight Python web framework for building web applications with minimal setup. [pynative](#)

362. What is Django?

Full-featured web framework following MVT pattern with ORM, admin interface, authentication. [pynative](#)

363. What is FastAPI?

Modern, fast web framework for building APIs with automatic documentation and type checking. [pynative](#)

364. What is WSGI?

Web Server Gateway Interface, standard for Python web applications and servers. [pynative](#)

365. What is ASGI?

Asynchronous Server Gateway Interface for async Python web applications. [pynative](#)

366. What is request-response cycle?

Client sends HTTP request, server processes and returns HTTP response. [pynative](#)

367. What are HTTP methods?

GET (retrieve), POST (create), PUT (update), DELETE (remove), PATCH (partial update). [pynative](#)

368. What is routing?

Mapping URLs to handler functions/views in web frameworks. [pynative](#)

369. What are templates?

HTML files with placeholders for dynamic content, using Jinja2 in Flask/Django. [pynative](#)

370. What is ORM?

Object-Relational Mapping translates between database tables and Python objects. [pynative](#)

371. What is SQLAlchemy?

Popular Python ORM for database interaction with support for multiple databases. [pynative](#)

372. What is Django ORM?

Built-in ORM in Django for database operations using Python classes and methods. [pynative](#)

373. What is middleware?

Software layer processing requests/responses between client and application. [pynative](#)

374. What is session management?

Maintaining user state across multiple requests using cookies or server-side storage. [pynative](#)

375. What is authentication vs authorization?

Authentication verifies identity, authorization determines permissions. [pynative](#)

376. What is REST API?

Representational State Transfer, architectural style for web services using HTTP. [pynative](#)

377. What are REST principles?

Stateless, client-server, cacheable, uniform interface, layered system. [pynative](#)

378. What is JSON Web Token (JWT)?

Compact token for authentication containing encoded JSON payload signed for integrity. [pynative](#)

379. What is CORS?

Cross-Origin Resource Sharing, mechanism allowing requests from different domains. [pynative](#)

380. What is web scraping?

Extracting data from websites using libraries like BeautifulSoup, Scrapy, Selenium.[pynative](#)

Data Science and Analytics**381. What is pandas?**

Python library for data manipulation and analysis using DataFrames and Series.[pynative](#)

382. What is DataFrame?

Two-dimensional labeled data structure with columns of potentially different types.[pynative](#)

383. What is Series?

One-dimensional labeled array in pandas.[pynative](#)

384. How to read CSV in pandas?

`pd.read_csv('file.csv')` loads CSV into DataFrame.[pynative](#)

385. What is data cleaning?

Handling missing values, duplicates, outliers, type conversion, normalization.[upesonline](#)

386. How to handle missing values?

`df.dropna()`, `df.fillna(value)`, `df.interpolate()`, imputation strategies.[upesonline](#)

387. What is data aggregation?

Combining data using `groupby()`, `agg()`, `pivot_table()` for summary statistics.[pynative](#)

388. What is data merging?

Combining DataFrames using `merge()`, `join()`, `concat()` operations.[upesonline](#)

389. What is data pivoting?

Reshaping data using `pivot()`, `pivot_table()`, `melt()` for analysis.[upesonline](#)

390. What is matplotlib?

Python plotting library for creating static, animated, and interactive visualizations.[pynative](#)

391. What is seaborn?

Statistical data visualization library built on matplotlib with high-level interface.[pynative](#)

392. What is scikit-learn?

Machine learning library providing classification, regression, clustering algorithms.[pynative](#)

393. What is supervised learning?

Learning from labeled data to predict outcomes: classification, regression.[pynative](#)

394. What is unsupervised learning?

Learning patterns from unlabeled data: clustering, dimensionality reduction.[pynative](#)

395. What is train-test split?

Dividing data into training set (model building) and test set (evaluation).[upesonline](#)

396. What is cross-validation?

Technique for assessing model performance by training on multiple data splits. [pynative](#)

397. What is feature engineering?

Creating new features from existing data to improve model performance. [pynative](#)

398. What is normalization vs standardization?

Normalization scales to, standardization centers with mean=0, std=1. [geeksforgeeks+1](#)

399. What is overfitting vs underfitting?

Overfitting: too complex, fits training data too well. Underfitting: too simple, poor fit. [pynative](#)

400. What is regularization?

Techniques preventing overfitting by penalizing model complexity: L1, L2 regularization. [pynative](#)

Advanced Python Features**401. What is type hinting?**

Python 3.5+ feature adding type annotations for better code documentation and tools. [pynative](#)

402. How to use type hints?

python

```
def greet(name: str) -> str:  
    return f"Hello, {name}"
```

403. What is typing module?

Provides type hint support: List, Dict, Optional, Union, Callable, TypeVar. [pynative](#)

404. What is Optional type?

Indicates value can be specified type or None: Optional[int] equals Union[int, None]. [github](#)

405. What is Union type?

Indicates value can be one of several types: Union[int, str] [github](#)

406. What is Callable type?

Type hint for functions: Callable[[int, str], bool] means function taking int, str, returning bool. [pynative](#)

407. What is Generic type?

Creating generic classes and functions using TypeVar for type flexibility. [pynative](#)

408. What is Protocol (structural subtyping)?

Defines interface based on methods/attributes, enabling duck typing with type checking. [pynative](#)

409. What is dataclasses?

Python 3.7+ decorator automatically generating special methods for classes:

python

```
from dataclasses import dataclass
```

```
@dataclass
```

```
class Point:
```

```
    x: int
```

```
    y: int
```

410. What are named tuples vs dataclasses?

Named tuples are immutable, dataclasses are mutable by default with more features. [pynative](#)

411. What is match-case (structural pattern matching)?

Python 3.10+ feature for pattern matching similar to switch statements:

python

```
match value:
```

```
    case 1:
```

```
        print("one")
```

```
    case _:
```

```
        print("other")
```

412. What is walrus operator (:=)?

Python 3.8+ assignment expression: if (n := len(data)) > 10: [github](#)

413. What are f-string expressions?

f-strings support expressions: f"{x:.2f}", f"{func()}", f"{obj.attr}" [datacamp](#)

414. What is positional-only parameters?

Parameters before / must be positional: def func(a, b, /, c): pass [github](#)

415. What is keyword-only parameters?

Parameters after * must be keyword: def func(a, *, b, c): pass [github](#)

416. What is `__future__` imports?

Enable future features in current version: from `__future__` import annotations. [github](#)

417. What is `__main__` guard?

python

```
if __name__ == "__main__":  
    main()
```

Ensures code runs only when script executed directly. [inttrvu](#)

418. What is virtual environment best practices?

Create per-project, use requirements.txt, don't commit venv directory, document Python version. [pynative](#)

419. What is requirements.txt?

File listing project dependencies for pip install -r requirements.txt. [pynative](#)

420. What is setup.py?

Script for distributing Python packages, defining metadata and dependencies. [pynative](#)

Database Operations

421. What is SQLite?

Lightweight embedded database, included with Python via sqlite3 module. [pynative](#)

422. How to connect to database?

python

```
import sqlite3
```

```
conn = sqlite3.connect('database.db')
```

```
cursor = conn.cursor()
```

[.pynative](#)

423. How to execute SQL queries?

python

```
cursor.execute("SELECT * FROM table")
```

```
results = cursor.fetchall()
```

424. What is SQL injection?

Security vulnerability when user input inserted directly into SQL queries. Use parameterized queries. [pynative](#)

425. How to prevent SQL injection?

Use placeholders: `cursor.execute("SELECT * FROM users WHERE id=?", (user_id,))`. [github](#)

426. What is transaction?

Group of operations executed as single unit, supporting commit and rollback. [pynative](#)

427. What is database normalization?

Organizing data to reduce redundancy: 1NF, 2NF, 3NF, BCNF forms. [pynative](#)

428. What is connection pooling?

Reusing database connections to improve performance and resource management. [pynative](#)

429. What are database indexes?

Data structures improving query speed at cost of slower writes and more storage. [pynative](#)

430. What is ACID properties?

Atomicity, Consistency, Isolation, Durability - database transaction guarantees. [pynative](#)

Security**431. What are common security vulnerabilities?**

SQL injection, XSS, CSRF, authentication bypass, insecure deserialization. [pynative](#)

432. What is password hashing?

One-way transformation of passwords using algorithms like bcrypt, PBKDF2, Argon2. [pynative](#)

433. How to hash passwords in Python?

```
python
```

```
import bcrypt
```

```
hashed = bcrypt.hashpw(password.encode(), bcrypt.gensalt())
```

434. What is encryption vs hashing?

Encryption is reversible (with key), hashing is one-way transformation. [pynative](#)

435. What is cryptography module?

Python library providing cryptographic recipes and primitives. [pynative](#)

436. What is secrets module?

Generates cryptographically strong random numbers for security: `secrets.token_hex()`. [github](#)

437. What is environment variables for secrets?

Store sensitive data in environment variables, not in code: `os.environ.get('API_KEY')`. [github](#)

438. What is input validation?

Verifying user input meets expected format, type, and constraints before processing. [pynative](#)

439. What is sanitization?

Cleaning user input to prevent injection attacks by removing/escaping dangerous characters.[pynative](#)

440. What is HTTPS and SSL/TLS?

Secure HTTP using encryption via SSL/TLS certificates for data transmission security.[pynative](#)

Logging and Monitoring**441. What is logging?**

Recording program events using logging module for debugging and monitoring.[pynative](#)

442. What are logging levels?

DEBUG, INFO, WARNING, ERROR, CRITICAL in increasing severity.[pynative](#)

443. How to configure logging?

python

import logging

logging.basicConfig(level=logging.INFO,

format='%(asctime)s - %(levelname)s - %(message)s')

444. What is logger vs handler?

Logger records messages, handler sends messages to destinations (file, console, network).[pynative](#)

445. What is log formatting?

Customizing log message format with timestamp, level, module, message, etc..[pynative](#)

446. What is log rotation?

Automatically creating new log files based on size or time to manage disk space.[pynative](#)

447. What is structured logging?

Logging events as structured data (JSON) instead of text for easier parsing.[pynative](#)

448. What is application monitoring?

Tracking application performance, errors, and usage metrics in production.[pynative](#)

449. What is error tracking?

Systems like Sentry capturing and aggregating application errors for debugging.[pynative](#)

450. What is distributed tracing?

Tracking requests across multiple services in microservices architecture.[pynative](#)

Packaging and Deployment

451. What is Python package?

Distributable collection of modules organized in directory with `__init__.py`. [github](#)

452. What is PyPI?

Python Package Index, repository for distributing and installing Python packages. [pynative](#)

453. What is wheel?

Built package format (.whl) for faster installation than source distributions. [pynative](#)

454. What is source distribution?

Package format containing source code, requiring compilation during installation. [pynative](#)

455. How to create Python package?

Structure with `setup.py`, include modules, use `python setup.py sdist bdist_wheel`. [github](#)

456. What is Poetry?

Modern dependency management and packaging tool with better resolver than `pip`. [pynative](#)

457. What is pipenv?

Tool combining `pip` and `virtualenv` for dependency management with `Pipfile`. [pynative](#)

458. What is Docker for Python?

Containerizing Python applications for consistent deployment across environments. [pynative](#)

459. What is virtual environment activation?

Windows: `venv\Scripts\activate`, Linux/Mac: `source venv/bin/activate`. [github](#)

460. What is continuous integration (CI)?

Automatically testing code changes using services like GitHub Actions, Travis CI. [pynative](#)

Advanced Problem Solving**461. Implement LRU Cache:**

python

```
from collections import OrderedDict
```

```
class LRUCache:
```

```
    def __init__(self, capacity):
```

```
        self.cache = OrderedDict()
```

```
        self.capacity = capacity
```

```
    def get(self, key):
```

```
        if key not in self.cache:
```

```
    return -1

    self.cache.move_to_end(key)

    return self.cache[key]
```

```
def put(self, key, value):

    if key in self.cache:

        self.cache.move_to_end(key)

    self.cache[key] = value

    if len(self.cache) > self.capacity:

        self.cache.popitem(last=False)
```

462. Median of two sorted arrays:

python

```
def find_median_sorted_arrays(nums1, nums2):

    if len(nums1) > len(nums2):

        nums1, nums2 = nums2, nums1

    m, n = len(nums1), len(nums2)

    left, right = 0, m

    while left <= right:

        partition1 = (left + right) // 2

        partition2 = (m + n + 1) // 2 - partition1

        max_left1 = float('-inf') if partition1 == 0 else nums1[partition1 - 1]
        min_right1 = float('inf') if partition1 == m else nums1[partition1]
        max_left2 = float('-inf') if partition2 == 0 else nums2[partition2 - 1]
        min_right2 = float('inf') if partition2 == n else nums2[partition2]
```

```

if max_left1 <= min_right2 and max_left2 <= min_right1:
    if (m + n) % 2 == 0:
        return (max(max_left1, max_left2) + min(min_right1, min_right2)) / 2
    else:
        return max(max_left1, max_left2)
elif max_left1 > min_right2:
    right = partition1 - 1
else:
    left = partition1 + 1

```

463. Longest palindromic substring:

python

```

def longest_palindrome(s):
    def expand_around_center(left, right):
        while left >= 0 and right < len(s) and s[left] == s[right]:
            left -= 1
            right += 1
        return right - left - 1

```

start = end = 0

```

for i in range(len(s)):
    len1 = expand_around_center(i, i)
    len2 = expand_around_center(i, i + 1)
    max_len = max(len1,

```

1. <https://www.geeksforgeeks.org/python/python-interview-questions/>
2. <https://www.datacamp.com/blog/top-python-interview-questions-and-answers>
3. <https://www.interviewbit.com/python-interview-questions/>
4. <https://pynative.com/python/interview-questions/>
5. <https://inttrvu.ai/basic-interview-questions-on-python/>

6. <https://upesonline.ac.in/blog/python-interview-questions-basics-and-advanced>
7. [https://github.com/Tanu-N-Prabhu/Python/blob/master/Python%20Coding%20Interview%20Prep/Python%20Coding%20Interview%20Questions%20\(Beginner%20to%20Advanced\).md](https://github.com/Tanu-N-Prabhu/Python/blob/master/Python%20Coding%20Interview%20Prep/Python%20Coding%20Interview%20Questions%20(Beginner%20to%20Advanced).md)
8. <https://pynative.com/python-oop-interview-questions/>
9. <https://www.hirist.tech/blog/top-25-python-oops-interview-question-2024/amp/>
10. <https://www.hipeople.io/interview-questions/advanced-python-interview-questions>
11. <https://www.terminal.io/blog/15-python-interview-questions-for-hiring-python-engineers>
12. <https://www.java-success.com/01-python-iterators-generators-decorators-tutorial/>
13. <https://github.com/david-legend/python-algorithms>
14. <https://www.vinsys.com/blog/python-interview-questions-and-answers>
15. <https://realpython.com/python-practice-problems/>
16. <https://www.ashokit.in/blog/top-25-python-interview-questions-and-answers>
17. <https://www.indiabix.com/technical/python/decorators/>
18. https://www.youtube.com/watch?v=6TreJMPX_M8